IMPLEMENTACION DE FUNSION PROPIA LLAMADA CADENA

cadena.h

```h
#ifndef CADENA_H
#define CADENA_H

#include <iostream>

class MonitorRecursos; // Declaración adelantada

class Cadena
{
private:
    char* datos;
    int longitud;

public:
    // Constructores y destructor
    Cadena();
    Cadena(const char* str);
    Cadena(const Cadena& otra);
    ~Cadena();

    // Getters
    int getLongitud() const;
    const char* c_str() const;

    // Métodos solicitados en el diagrama UML
    void asignar(const char* str);
    bool comparar(const Cadena& otra) const;

    // Sobrecarga de operadores (cumple con el requisito de al menos 2 operadores)
    Cadena& operator=(const Cadena& otra);
    bool operator==(const Cadena& otra) const;
    bool operator!=(const Cadena& otra) const;

    // Métodos utilitarios para rutas, comparaciones de fechas e I/O
    bool contiene(const Cadena& sub) const;
    int compararLexicografico(const Cadena& otra) const; // Útil para ordenar y buscar fechas
    friend std::ostream& operator<<(std::ostream& os, const Cadena& c);
    friend std::istream& operator>>(std::istream& is, Cadena& c);
};

#endif // CADENA_H
```

la clase Cadena se diseñó e implementó como un Tipo de Dato Abstracto (TDA) utilitario fundamental para dar cumplimiento estricto a la restricción del enunciado de **no emplear la biblioteca estándar (cero STL / sin std::string)** en todo el desarrollo del sistema

1. ¿Cómo está modelada en memoria?

"A nivel de atributos privados, la clase no utiliza arreglos estáticos de tamaño fijo para evitar el desperdicio de memoria. Encapsula únicamente dos variables:

- *Un puntero char* datos que apunta a un bloque contiguo asignado dinámicamente en el **Heap**, terminado en el carácter nulo ('\0').*
- *Un entero int longitud, que almacena el tamaño útil de la cadena sin el terminador. Esto nos permite responder a consultas de longitud mediante getLongitud() en tiempo constante $O(1)$, sin necesidad de recorrer el arreglo repetidamente."*

2. Ciclo de vida y gestión segura de memoria (Regla de los Tres)

*"Dado que Cadena administra memoria dinámica directamente con punteros, implementamos de manera estricta la **Regla de los Tres** para garantizar la integridad referencial y prevenir fallos críticos de memoria:*

1. ***Constructor:** Reserva con new char[longitud + 1] el espacio exacto para almacenar el texto entrante.*
2. ***Destructor:** Ejecuta delete[] datos tan pronto el objeto sale de ámbito o se destruye, eliminando cualquier riesgo de fuga de memoria (memory leaks).*

3. ***Constructor de copia y operador de asignación (operator=):*** *Realizan copias profundas (deep copy). Es decir, al asignar o clonar una cadena (por ejemplo, al crear un Usuario, transferir un Mensaje o asignar rutas de fotos), se reserva un nuevo bloque en memoria para evitar punteros colgantes (dangling pointers) o colapsos por doble liberación (double free)."*

3. Sobrecarga de operadores y funcionalidad en la red social

"Para que Cadena se comportara de forma natural y transparente con el resto de entidades del sistema, sobrecargamos varios operadores clave:

- ***operator== y operator!=:*** *Permiten la comparación de texto carácter por carácter, esencial en el proceso de autenticación de credenciales (usuario y contraseña) y en la búsqueda de nodos en el grafo de amistades.*
- ***compararLexicografico():*** *Implementa una resta de valores ASCII entre caracteres. Esta función es la base técnica del* ***Escenario de Búsqueda de Mensajes****, ya que permite evaluar si una fecha en formato ISO (AAAA-MM-DD) se encuentra contenida dentro de un rango cronológico determinado sin conversiones complejas.*
- ***operator<< y operator>> (funciones friend):*** *Sobrecargan los flujos estándar para permitir imprimir cadenas directamente en pantalla mediante std::cout y leer o persistir los datos de manera limpia en los 5 archivos planos (.txt)."*